

Splunk Process Tree Detection Lab

Windows Event ID 4688 Investigation and Parent-Child Process Analysis

Overview

This project documents a hands-on Splunk detection lab focused on monitoring Windows process creation events and reconstructing parent-child process execution chains using Windows Event ID 4688.

The objective of the lab was to understand how Security Operations Center (SOC) analysts investigate suspicious process activity and detect abnormal execution behavior through Windows telemetry.

Using Splunk Enterprise, Sysmon, and Windows Security logs, I analyzed:

- Process creation events
- Parent-child process relationships
- PowerShell execution behavior
- Hidden PowerShell activity
- Suspicious execution chains
- Windows Event ID 4688 telemetry

The lab environment consisted of virtual machines configured for centralized log collection and security monitoring.

Lab Environment

Virtual Machines Used

Ubuntu VM

Used as the Splunk Enterprise server.

Windows VM

Used for generating Windows process creation events and security telemetry.

Kali Linux VM

Used as part of the broader SOC lab environment.

Technologies Used

Tool	Purpose
Splunk Enterprise	SIEM and log analysis
Sysmon	Endpoint telemetry collection
Splunk Universal Forwarder	Windows log forwarding
Windows Security Logs	Process creation monitoring
PowerShell	Execution chain testing
CMD	Parent process simulation
VirtualBox / VMware	Virtualization

Understanding Windows Event ID 4688

Windows Event ID 4688 represents:

“A New Process Has Been Created”

This event is one of the most important telemetry sources used by SOC analysts and threat hunters because it provides visibility into:

- Process execution
- Parent-child process relationships
- Command-line arguments
- Execution chains

- Potential attacker activity

By analyzing process creation events, analysts can reconstruct attacks and identify suspicious behavior.

LAB 1 — CMD → PowerShell → Notepad Execution Chain

Objective

The objective of this exercise was to observe how Windows logs record parent-child process relationships when one process spawns another.

The following command was executed from CMD:

```
powershell -Command "notepad.exe"
```

This generated the following execution chain:

```
cmd.exe
├── powershell.exe
│   └── notepad.exe
```

SPL Query Used

```
index=wineventlog EventCode=4688 notepad.exe
| table _time Account_Name Creator_Process_Name New_Process_Name
Process_Command_Line
| sort - _time
```

Investigation Findings

Using Splunk, I analyzed:

- Parent process names
- Child process names
- Process command lines
- Execution timestamps
- Process creation relationships

The investigation revealed:

Parent Process

cmd.exe

Child Process

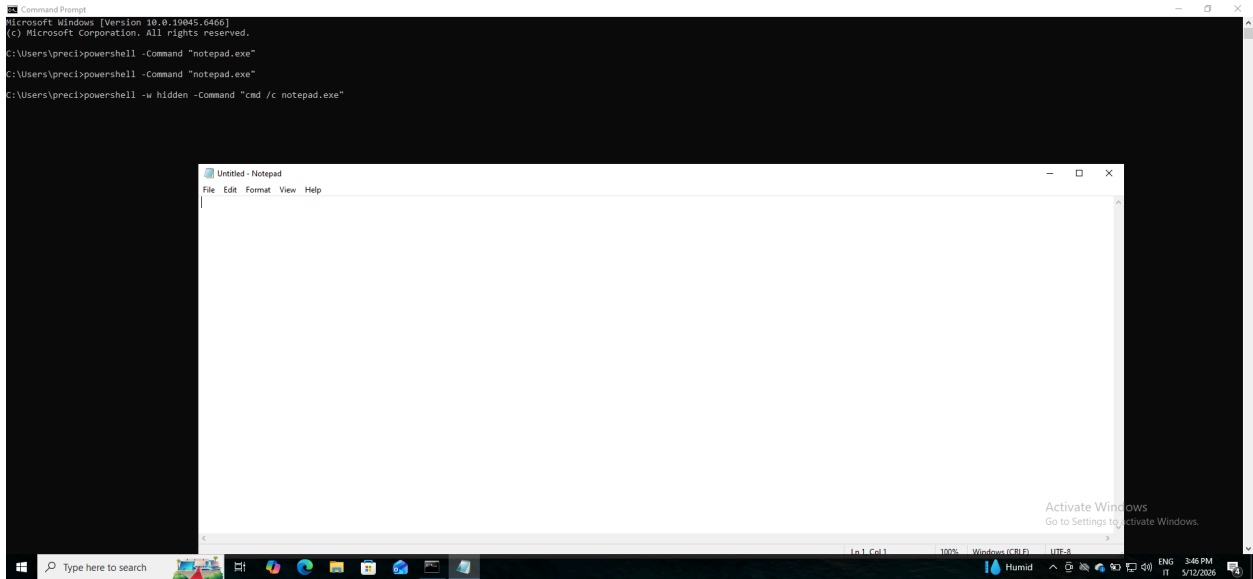
powershell.exe

followed by:

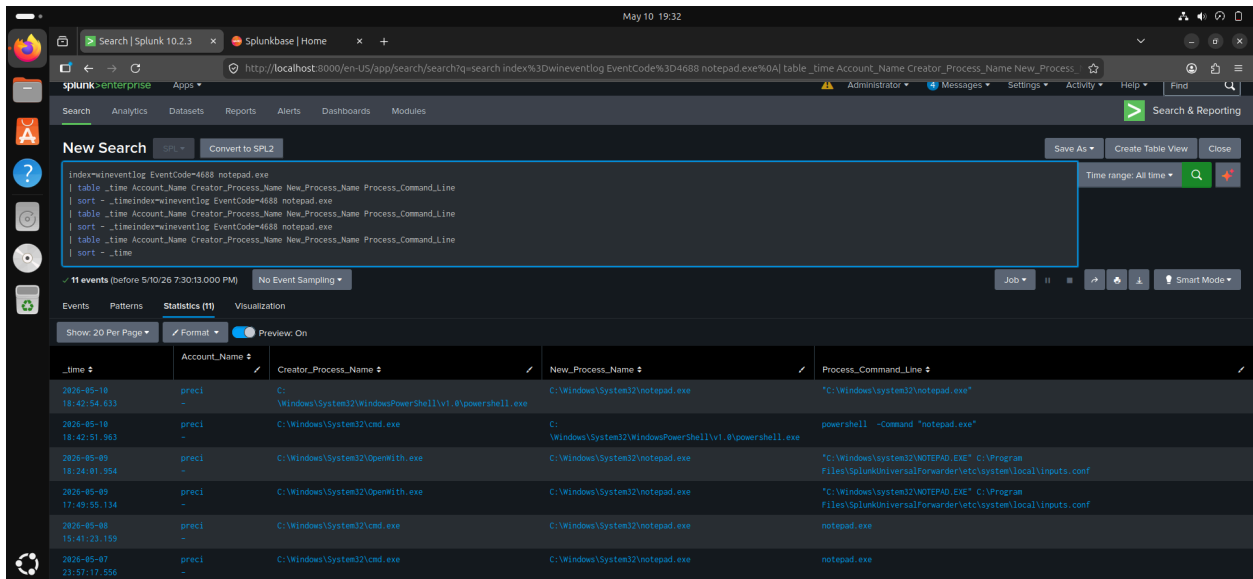
notepad.exe

This demonstrated how processes spawn additional processes during execution and how these relationships are recorded within Windows telemetry.

Screenshot 1 — LAB 1 EventCode 4688 Creation



Screenshot 2 — Notepad Spawning 4688 Event Lab



Why Process Trees Matter in SOC Investigations

Attackers rarely execute only a single isolated process.

Instead, malicious activity often appears as process chains such as:

```
Office Application
├── powershell.exe
│   ├── cmd.exe
│   │   ├── rundll32.exe
│   │   │   └── malicious payload
```

SOC analysts reconstruct attacks by analyzing process trees and identifying abnormal parent-child relationships.

This helps detect:

- Malicious PowerShell execution
 - Living Off The Land Binaries (LOLBins)
 - Script-based attacks
 - Malware launch chains
 - Persistence techniques
 - Privilege escalation attempts
-

LAB 2 — Hidden PowerShell Execution Chain

Objective

The purpose of this exercise was to investigate hidden PowerShell execution and identify suspicious process behavior.

The following command was executed:

```
powershell -w hidden -Command "cmd /c notepad.exe"
```

This launches PowerShell with a hidden window and spawns CMD, which then launches Notepad.

Investigation Questions

During the investigation, I analyzed:

- Which process executed first
 - Which process became hidden
 - Which process remained visible
 - Which process appeared suspicious
 - How the execution chain differed from Lab 1
-

SPL Query Used

```
index=wineventlog EventCode=4688  
| table _time Account_Name Creator_Process_Name New_Process_Name  
Process_Command_Line  
| sort - _time
```

Investigation Findings

The execution chain appeared similar to:

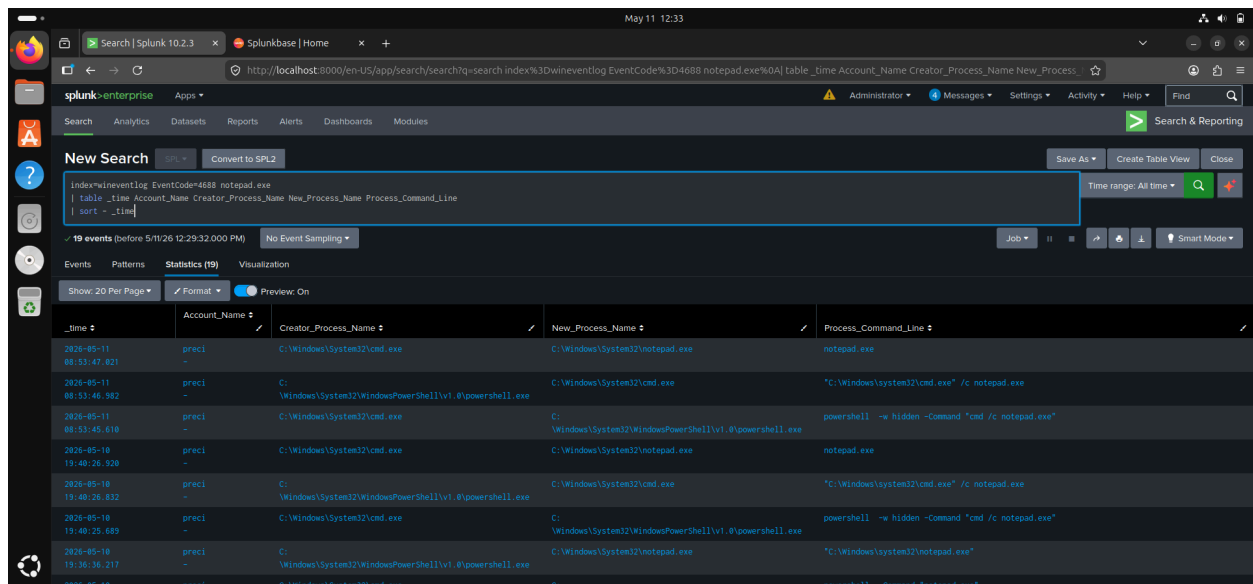
```
powershell.exe (hidden)  
└── cmd.exe  
    └── notepad.exe
```

Key observations included:

- PowerShell executed using hidden window arguments
- CMD was spawned through PowerShell
- Notepad was launched through CMD
- Hidden PowerShell execution can indicate suspicious behavior

This is important because attackers frequently use hidden PowerShell execution to avoid user visibility and reduce detection.

Screenshot 3 — LAB 2 EventCode 4688 Detection



The screenshot shows the Splunk Enterprise web interface. A search bar at the top contains the query: `index=wineventlog EventCode=4688 notepad.exe | table _time Account_Name Creator_Process_Name New_Process_Name Process_Command_Line | sort - _time`. Below the search bar, the results are displayed in a table format. The table has 5 columns: `_time`, `Account_Name`, `Creator_Process_Name`, `New_Process_Name`, and `Process_Command_Line`. The results show a sequence of process creations, including `cmd.exe` and `powershell.exe` spawning `notepad.exe`.

_time	Account_Name	Creator_Process_Name	New_Process_Name	Process_Command_Line
2025-05-11 08:53:47.021	prec1	C:\Windows\System32\cmd.exe	C:\Windows\System32\notepad.exe	notepad.exe
2025-05-11 08:53:46.982	prec1	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\cmd.exe	"C:\Windows\system32\cmd.exe" /c notepad.exe
2025-05-11 08:53:42.618	prec1	C:\Windows\System32\cmd.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell -w hidden -Command "cmd /c notepad.exe"
2025-05-10 15:40:25.520	prec1	C:\Windows\System32\cmd.exe	C:\Windows\System32\notepad.exe	notepad.exe
2025-05-10 15:40:25.832	prec1	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\cmd.exe	"C:\Windows\system32\cmd.exe" /c notepad.exe
2025-05-10 15:40:25.689	prec1	C:\Windows\System32\cmd.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell -w hidden -Command "cmd /c notepad.exe"
2025-05-10 15:36:36.217	prec1	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\notepad.exe	"C:\Windows\system32\notepad.exe"
2025-05-10 15:36:36.217	prec1	C:\Windows\System32\cmd.exe	C:\Windows\System32\notepad.exe	powershell -Command "notepad.exe"

Skills and Knowledge Gained

Through this project, I gained practical experience in:

- Process creation monitoring
 - Windows Event ID 4688 analysis
 - Splunk SPL searches
 - Parent-child process investigation
 - Process tree reconstruction
 - PowerShell telemetry analysis
 - Hidden process execution analysis
 - Threat hunting concepts
 - SOC investigation workflows
-

Detection Relevance

These techniques are highly relevant in modern SOC environments because attackers commonly abuse:

- powershell.exe
- cmd.exe
- rundll32.exe
- mshta.exe
- regsvr32.exe
- wscript.exe

Monitoring abnormal process chains is an important blue-team detection strategy used to identify suspicious execution behavior.

Challenges Encountered

Some challenges encountered during the lab included:

- Understanding Windows process hierarchy
- Interpreting Event ID 4688 fields
- Correlating parent and child processes
- Understanding PowerShell execution flags
- Distinguishing normal versus suspicious activity

Resolving these challenges improved my investigative and analytical skills.

Future Improvements

Future improvements planned for this lab include:

- Creating Splunk dashboards for process monitoring
- Developing PowerShell detection rules
- Detecting encoded PowerShell commands
- Simulating additional attacker execution chains
- Investigating LOLBins in Windows
- Building process anomaly alerts

- Expanding endpoint telemetry collection
-

Conclusion

This lab improved my understanding of how SOC analysts investigate Windows process execution activity using Splunk.

By analyzing Event ID 4688 logs and reconstructing process trees, I gained practical experience in:

- Process monitoring
- Parent-child process analysis
- PowerShell investigation
- Threat hunting concepts
- Suspicious execution detection
- Security event correlation

This project strengthened my understanding of Windows telemetry and how process relationships are used to detect suspicious activity in SOC environments.

Author

Precious Anyanwu

Cybersecurity Learner | SOC Analyst Path | Splunk SIEM Practice